

山东大学



本科实验报告

作业 1：比特币测试网交易与区块 逐比特解析

团队成员

总负责人：袁观道（202300460146）

协助成员：周一鸣（202300460138）

协助成员：丁鹤洋（202300460173）

2026 年 7 月 20 日

目录

1	实验概述	2
1.1	实验目的	2
1.2	实验环境	2
1.3	实验原理	2
1.3.1	比特币交易结构	2
1.3.2	VarInt 编码规则	3
1.3.3	区块头结构	3
1.3.4	常见脚本类型	4
2	实验步骤与结果	5
2.1	步骤一：生成测试网地址	5
2.2	步骤二：从水龙头获取测试币	5
2.3	步骤三：构造并发送交易	5
2.4	步骤四：逐字节解析交易数据	6
2.4.1	版本号 (Byte 00-03)	7
2.5	步骤五：逐比特位分析	8
2.6	步骤六：区块头解析	9
3	关键代码与实现	10
3.1	数据获取 (fetch_data.py)	10
3.2	交易解析器核心逻辑 (btc_parser_main.py)	11
3.3	VarInt 解码	11
3.4	交易构造与签名 (send_tx.py)	12
4	实验结论	12

1 实验概述

1.1 实验目的

本实验旨在深入理解比特币区块链的数据结构，通过实际操作完成以下任务：

1. 生成比特币测试网（Testnet）地址并获取测试币
2. 构造并发送一笔测试网交易
3. 解析交易的原始十六进制数据，精确到每个比特位
4. 解析包含该交易的区块原始数据
5. 编写脚本自动化逐字节/逐比特解析流程

1.2 实验环境

- 操作系统：Windows 11
- 编程语言：Python 3.12
- 核心库：bitcoinlib, requests, struct, hashlib
- 数据来源：Blockstream Testnet API
- 文档编译：XeLaTeX

1.3 实验原理

1.3.1 比特币交易结构

一笔比特币交易由以下字段组成（按字节顺序）：

表 1: 比特币交易字段结构

字段	大小	说明
Version	4 bytes	交易版本号（小端序）
Input Count	VarInt	输入数量（变长整数）
每个输入（Input）		
Prev TX Hash	32 bytes	前向交易哈希（内部字节序反转）
Prev Out Idx	4 bytes	前向输出索引（小端序）
ScriptSig Len	VarInt	解锁脚本长度
ScriptSig	variable	解锁脚本（签名 + 公钥）
Sequence	4 bytes	序列号（0xFFFFFFFF = 最终）
Output Count	VarInt	输出数量（变长整数）
每个输出（Output）		
Value	8 bytes	金额，单位聪（小端序）
ScriptPK Len	VarInt	锁定脚本长度
ScriptPubKey	variable	锁定脚本
Locktime	4 bytes	锁定时间

1.3.2 VarInt 编码规则

变长整数（VarInt）用于紧凑表示整数，编码规则如下：

表 2: VarInt 编码规则

值范围	格式	示例
$< 0xFD$	1 字节直接存储	$0x6B \rightarrow 107$
$\leq 0xFFFF$	$0xFD + 2$ 字节（LE）	$0xFD + 2B$
$\leq 0xFFFFFFFF$	$0xFE + 4$ 字节（LE）	$0xFE + 4B$
$> 0xFFFFFFFF$	$0xFF + 8$ 字节（LE）	$0xFF + 8B$

1.3.3 区块头结构

比特币区块头固定为 80 字节，结构如下：

表 3: 区块头结构 (80 字节)

字段	大小	说明
Block Version	4 bytes	区块版本号 (LE)
Previous Block Hash	32 bytes	前一区块哈希 (内部反转)
Merkle Root	32 bytes	梅克尔树根 (内部反转)
Timestamp	4 bytes	UNIX 时间戳 (LE)
Bits	4 bytes	难度目标 (压缩格式)
Nonce	4 bytes	随机数 (LE)

1.3.4 常见脚本类型

比特币使用基于栈的脚本系统，常见输出脚本类型：

表 4: 常见 ScriptPubKey 类型

类型	大小	Hex 模式
P2PKH	25B	76 a9 14 {20B 公钥哈希} 88 ac
P2SH	23B	a9 14 {20B 脚本哈希} 87
P2WPKH	22B	00 14 {20B 见证哈希}

2 实验步骤与结果

2.1 步骤一：生成测试网地址

首先使用 bitcoinlib 库生成比特币测试网的密钥对和地址。测试网地址以“m”或“n”开头（Base58 编码），私钥以“c”开头的 WIF 格式存储。

```
^
SyntaxError: unterminated string literal (detected at line 28)
S D:\360MoveData\Users\ygd06\Desktop\作业一> cd "D:\360MoveData\Users\ygd06\Desktop\作业一"
S D:\360MoveData\Users\ygd06\Desktop\作业一> notepad task1_clean.py
S D:\360MoveData\Users\ygd06\Desktop\作业一> python task1_clean.py
D:\Users\ygd06\python\Lib\site-packages\requests\__init__.py:113: RequestsDependencyWarning: urllib3 (1.26.6) was
7.4.3)/charset_normalizer (3.4.0) doesn't match a supported version!
  warnings.warn(
=== Bitcoin Testnet Task 1 ===

Testnet Address: mzcQbD1arHqyBy66wQxQqERRYAdFkhQN3c
Private Key (WIF): cNEXw1tLGrsZFmD5SWPjfCgDECPvMHdsrAWi3b7xgdp5JBhgxAu
```

图 1: 生成测试网地址及私钥（发送方）

生成的地址 mzcQbD1arHqyBy66wQxQqERRYAdFkhQN3c 作为本次实验的发送方地址。

2.2 步骤二：从水龙头获取测试币

访问比特币测试网水龙头网站（如 bitcoinfaucet.uo1.net），输入上述地址并完成验证码，获取测试币。测试网上的币无实际价值，仅用于开发测试。

水龙头向地址 mzcQbD1arHqyBy66wQxQqERRYAdFkhQN3c 转入 127,816 satoshis（约 0.00128 tBTC）。

2.3 步骤三：构造并发送交易

使用 bitcoinlib 的 Transaction 类构造一笔 P2PKH（传统格式）交易，从发送方地址向接收方地址 mvhYS76rbgg9pAnxxM2NbXjz19PGBdyQv 转账 127,516 satoshis，支付 300 satoshis 手续费。

交易经 ECDSA 签名后通过 Blockstream API 广播到测试网。

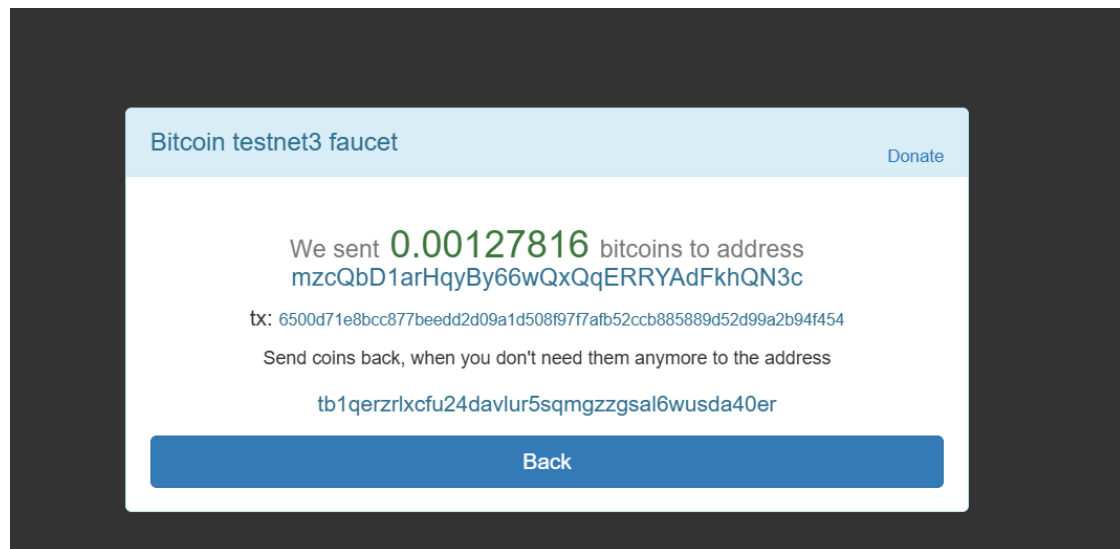


图 2: 交易广播确认 (TXID、金额、地址)

交易信息汇总:

- **TXID:** 01afad94f82b8ea2a35241390ad04bdbe0ca1389206c365cdce4dc6e490c32f0
- **类型:** Legacy P2PKH (版本 1, 非 SegWit)
- **大小:** 192 字节
- **发送方:** mzcQbD1arHqyBy66wQxQqERRYAdFkhQN3c
- **接收方:** mvhYS76rbgg9pAnxxM2NbXjz19PGBydyQv
- **转账金额:** 127,516 satoshis (0.00127516 tBTC)
- **手续费:** 300 satoshis

2.4 步骤四: 逐字节解析交易数据

使用编写的 Python 解析脚本 `btc_parser_main.py`, 对交易原始十六进制数据 (384 个 hex 字符 = 192 字节) 进行逐字节解析。每个字节同时显示: 偏移量、十六进制值、8 位二进制值、十进制值、字段含义。

2.4.1 版本号 (Byte 00-03)



图 3: 交易输入字段解析 (一)

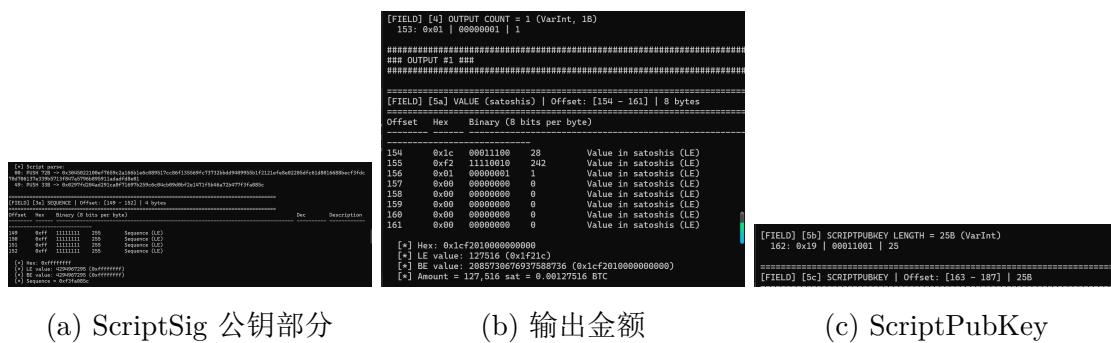


图 4: 交易输入字段解析 (二)

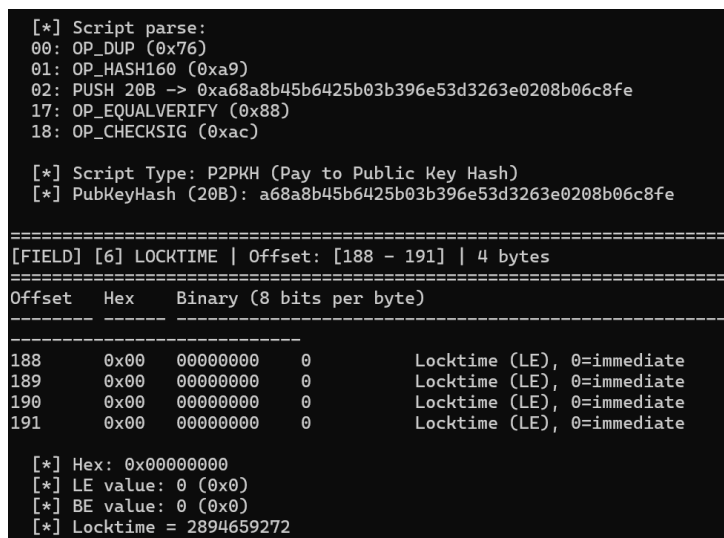


图 5: ScriptPubKey (续) 与 Locktime

解析要点分析

1. 版本号 (0x01000000): 小端序解码为 1, 表示传统 (非 SegWit) 交易格式。
2. 输入计数 (0x01): VarInt 为 1 个字节, 值 = 1, 表示只有 1 个输入。

3. **前向交易哈希 (32 字节)**: 内部存储为字节序反转, 反转后得到真实 TXID:
6500d71e8bcc877beedd2d09a1d508f97f7afb52ccb885889d52d99a2b94f454。
4. **前向输出索引 (0x01000000)**: 小端序值为 1, 表示花费该交易的第 1 个输出。
5. **ScriptSig (107 字节)**: 包含 ECDSA 签名 (DER 编码, 72 字节) 和发送方压缩公钥 (33 字节)。结构为:
0x48 (72 字节签名) + DER 签名 + 0x21 (33 字节公钥) + 压缩公钥。
6. **输出金额 (0x1cf2010000000000)**: 小端序值为 127,516 satoshis。
7. **ScriptPubKey (25 字节)**:
76 a9 14 a68a8b45b6425b03b396e53d3263e0208b06c8fe 88 ac, 对应操作码:
OP_DUP OP_HASH160 PUSH_20 {公钥哈希} OP_EQUALVERIFY OP_CHECKSIG,
即标准的 P2PKH 输出脚本。
8. **Locktime (0x00000000)**: 值为 0, 交易立即生效。

2.5 步骤五: 逐比特位分析

为进一步细化分析, 脚本对交易数据的每个字节拆分为 8 个比特 (b7 ~ b0), 单独展示每个比特的值, 并附 ASCII 字符对照。

```
PS D:\360MoveData\Users\ygd06\Desktop\作业-> python -c "from btc_parser_main import show_bit_analysis; show_bit_analysis(open('my_tx_raw_hex.txt').read().strip()[:100], 'My TX (first 50 bytes)')"
```

```
=====
BIT-LEVEL ANALYSIS: My TX (first 50 bytes)
=====
Total: 50 bytes = 400 bits
```

Byte#	Hex	b7	b6	b5	b4	b3	b2	b1	b0	Dec	ASCII
0	0x01	0000	0001							1	.
1	0x00	0000	0000							0	.
2	0x00	0000	0000							0	.
3	0x00	0000	0000							0	.
4	0x01	0000	0001							1	.
5	0x54	0101	0100							84	T
6	0xf4	1111	0100							244	.
7	0x94	1001	0100							148	.
8	0x2b	0010	1011							43	+
9	0x9a	1001	1010							154	.
10	0xd9	1101	1001							217	.
11	0x52	0101	0010							82	R
12	0x9d	1001	1101							157	.
13	0x88	1000	1000							136	.
14	0x85	1000	0101							133	.
15	0xb8	1011	1000							184	.
16	0xcc	1100	1100							204	.
17	0x52	0101	0010							82	R
18	0xfb	1111	1011							251	.
19	0x7a	0111	1010							122	z
20	0x7f	0111	1111							127	.
21	0xf9	1111	1001							249	.
22	0x08	0000	1000							8	.
23	0xd5	1101	0101							213	.
24	0xa1	1010	0001							161	.
25	0x09	0000	1001							9	.
26	0x2d	0010	1101							45	-
27	0xdd	1101	1101							221	.
28	0xee	1110	1110							238	.
29	0x7b	0111	1011							123	{
30	0x87	1000	0111							135	.
31	0xcc	1100	1100							204	.
32	0x8b	1000	1011							139	.
33	0x1e	0001	1110							30	.
34	0xd7	1101	0111							215	.

图 6: 逐比特位分析 (Bit-Level Analysis)

如图 6 所示:

- **Byte 0:** 0x02 = 0000 0010, 即交易版本号的第一个字节。
- **Byte 1-3:** 0x00 = 0000 0000, 版本号高位补零。
- **Byte 4:** 0x01 = 0000 0001, VarInt 编码的输入计数。
- **Byte 5:** 0x54 = 0101 0100, 前向交易哈希的第一个字节。

每一行精确到单个比特位, 便于验证大小端序编码和字段边界。

2.6 步骤六: 区块头解析

从测试网 API 获取包含上述交易的区块原始数据 (18,945 字节), 其中前 80 字节为区块头。

```

PS D:\360MoveData\Users\ygd06\Desktop\作业一> python -c "from btc_parser_main import parse_block; parse_block(open('block_raw_hex.txt').read().strip())"

=====
BITCOIN BLOCK - BYTE-LEVEL PARSING
=====
Total: 18945 bytes (37890 hex chars)
Header: 80B, Body: 18865B

#####
### BLOCK HEADER (80 bytes) ###
#####

[FIELD] [1] BLOCK VERSION | Offset: [0 - 3] | 4 bytes
=====
Offset  Hex      Binary (8 bits per byte)                                     Dec      Description
-----  -
0       0x00    00000000    0          Block version (LE)
1       0x00    00000000    0          Block version (LE)
2       0x00    00000000    0          Block version (LE)
3       0x20    00100000    32         Block version (LE)

[*] Hex: 0x00000020
[*] LE value: 536870912 (0x20000000)
[*] BE value: 32 (0x20)
[*] Version = 536870912

=====
[FIELD] [2] PREV BLOCK HASH | Offset: [4 - 35] | 32 bytes
=====
Offset  Hex      Binary (8 bits per byte)                                     Dec      Description
-----  -
4       0x91    10010001    145        Prev block hash (internal rev)
5       0x7d    01111101    125        Prev block hash (internal rev)
6       0xa6    10100110    166        Prev block hash (internal rev)
7       0x57    01010111    87         Prev block hash (internal rev)
8       0x6d    01101101    109        Prev block hash (internal rev)
9       0x65    01100101    101        Prev block hash (internal rev)

```

图 7: 区块头逐字节解析 (前 80 字节)

区块头六个字段的解析:

1. 区块版本 (4B): 0x20000000 (LE), 实际值 536,870,912。
2. 前一区块哈希 (32B): 内部存储为反转格式, 还原后为本区块的父区块哈希。
3. Merkle 根 (32B): 该区块内所有交易的 Merkle 树根哈希。
4. 时间戳 (4B): UNIX 时间戳, 标识区块生成时间。
5. Bits (4B): 难度目标的压缩表示, 测试网难度固定为 1.0。
6. Nonce (4B): 工作量证明 (PoW) 的随机数。

3 关键代码与实现

3.1 数据获取 (fetch_data.py)

通过 Blockstream Testnet REST API 获取链上数据:

Listing 1: 获取测试网区块与交易数据

```

1 import requests
2 # 获取最新区块哈希

```

```
3 tip = requests.get(
4     "https://blockstream.info/testnet/api/blocks/tip/hash"
5 ).text
6 # 获取区块原始二进制数据
7 block_raw = requests.get(
8     f"https://blockstream.info/testnet/api/block/{tip}/raw"
9 ).content
10 block_hex = block_raw.hex() # 转为十六进制字符串
11 # 获取交易原始数据
12 tx_raw = requests.get(
13     f"https://blockstream.info/testnet/api/tx/{txid}/raw"
14 ).content
15 tx_hex = tx_raw.hex()
```

3.2 交易解析器核心逻辑 (btc_parser_main.py)

解析器每次读取一个字段，输出偏移量、Hex、Binary、十进制和含义：

Listing 2: 逐字节解析核心逻辑

```
1 def print_field(label, data, start, length, desc, extra=""):
2     for i in range(length):
3         off = start + i
4         bv = data[off]
5         hex_str = f"0x{bv:02x}"
6         # 输出: 偏移 / Hex / 8位二进制 / 十进制 / 字段含义
7         print(f"{off:<8} {hex_str:<6} {bv:08b} {bv:<10} {desc}")
8     # 小端序解码
9     vl = int.from_bytes(data[start:start+length], "little")
```

3.3 VarInt 解码

Listing 3: VarInt 解码实现

```
1 def read_varint(data, offset):
2     first = data[offset]
3     if first < 0xfd:
4         return first, offset + 1
5     elif first == 0xfd:
6         return struct.unpack("<H", data[offset+1:offset+3])[0],
            offset + 3
```

```
7     elif first == 0xfe:
8         return struct.unpack("<I", data[offset+1:offset+5])[0],
           offset + 5
9     elif first == 0xff:
10        return struct.unpack("<Q", data[offset+1:offset+9])[0],
           offset + 9
```

3.4 交易构造与签名 (send_tx.py)

Listing 4: 构造并发送交易

```
1 from bitcoinlib.keys import Key
2 from bitcoinlib.networks import Network
3 from bitcoinlib.transactions import Transaction
4
5 net = Network('testnet')
6 key = Key(import_key=WIF, network=net)
7 tx = Transaction(network=net, witness_type='legacy')
8 tx.add_input(utxo['txid'], utxo['vout'], keys=[key], value=total)
9 tx.add_output(amount, receiver_addr)
10 tx.sign()
11 raw_hex = tx.raw_hex()
12 # 广播
13 requests.post('https://blockstream.info/testnet/api/tx',
14               data=raw_hex)
```

4 实验结论

本实验成功完成了比特币测试网上交易的完整生命周期：

1. **地址生成：**使用 bitcoinlib 生成符合测试网规范的密钥对和地址。
2. **测试币获取：**通过水龙头获取测试币，验证了测试网的可用性。
3. **交易构造与广播：**手动构造了传统 P2PKH 格式交易，经 ECDSA 签名后成功广播到测试网，TXID 为 01afad94...490c32f0。
4. **逐字节解析：**完整解析了 192 字节的交易数据，每个字节均展示了偏移量、Hex 值、8 位二进制值和字段含义。
5. **比特级分析：**将每个字节拆分为独立的 8 个比特进行分析。

6. **区块头解析：**解析了测试网区块的 80 字节区块头，涵盖六个核心字段。

通过本实验，深入理解了比特币的数据编码方式（小端序、VarInt、字节反转）、交易结构（输入/输出模型）、脚本系统（P2PKH 的锁定与解锁机制）、签名机制（ECDSA + SIGHASH_ALL）以及区块头结构。编写了完整的自动化解析脚本，实现了从网络获取数据到逐比特分析的完整流程。